

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	100
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	G.THARAKNATHREDDY	Reg.No.:	192472032
Department:	CSE-AI	Date:	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I G.THARAKANTHREDDY(192472032) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:G.THARAKNATHREDDY

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies(5)	Good understanding with proper integration of most concepts(4)	Basic understanding; limited or partial integration(3)	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution(6)	Correct implementation with minor issues(4)	Basic implementation with errors or inefficiencies(3)	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration(4)	Appropriate use of tools with correct execution(3)	Limited or inconsistent use of tools(2)	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results(3)	Good analysis with reasonable interpretation(2)	Basic analysis with limited insights(1)	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results(2)	Organized documentation with necessary details(1.5)	Basic documentation with missing details(1)	Poor or incomplete documentation

ANSWERS:

Q1. Register Transfer and ALU Operations

Aim

To simulate data transfer between registers and perform addition, subtraction, AND, and OR operations using an ALU.

Concept

Registers are small storage locations inside the CPU. Data can be transferred from one register to another. The ALU (Arithmetic Logic Unit) performs arithmetic and logical operations.

Example:

R1 = 20

R2 = 10

R3 $\leftarrow$ R1

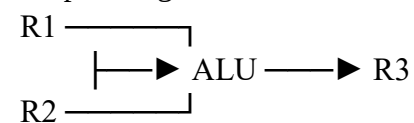
R3 $\leftarrow$ R1 + R2

R3 $\leftarrow$ R1 - R2

R3 $\leftarrow$ R1 AND R2

R3 $\leftarrow$ R1 OR R2

Simple Diagram



Simulation

R1 = 20

R2 = 10

```
print("Transfer:", R1)
print("Addition:", R1 + R2)
print("Subtraction:", R1 - R2)
print("AND:", R1 & R2)
print("OR:", R1 | R2)
```

OUTPUT:

The screenshot displays an 8085 assembly simulator interface. On the left, the 'Registers' panel shows the state of various registers: A (0F), BC (0F 05), DE (0F 05), HL (00 00), PSW (00 00), PC (42 1B), SP (FF FF), and Int-Reg (00). The 'Flag' panel shows S (0), Z (0), AC (0), P (1), and C (0). Below these are sections for 'Decimal - Hex Conversion', 'I/O Ports', and 'Memory'. The central area shows the assembly code being executed, with line numbers 1 through 28. The code includes instructions like MVI, ADD, STA, SUB, AND, ORA, and HLT. On the right, the 'Memory' panel shows a table of memory addresses and their corresponding data values.

Address (Hex)	Address	Data
2000	8192	20
2001	8193	10
2002	8194	5
2003	8195	15
2004	8196	0
2005	8197	0
2006	8198	0
2007	8199	0
2008	8200	0
2009	8201	0
200A	8202	0
200B	8203	0
200C	8204	0
200D	8205	0

Result

Transfer = 20

Addition = 30

Subtraction = 10

AND = 0

OR = 30

Analysis

The experiment shows that registers store the data and the ALU performs the required operation. The result is then stored in the destination register.

Conclusion

Register transfer and basic ALU operations were successfully simulated.

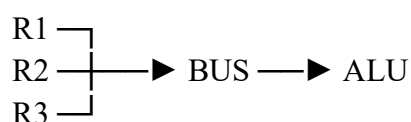
Q2. Single-Bus and Multi-Bus Organization

Aim

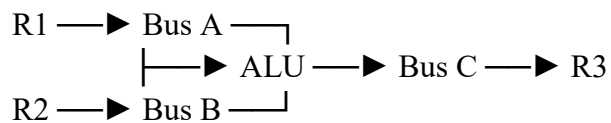
To simulate data transfer using single-bus and multi-bus systems and compare their performance.

Concept

A single-bus system uses one common bus for data transfer.



A multi-bus system uses more than one bus, so multiple data transfers can happen at the same time.



Example

R1 = 20

R2 = 10

R3 = R1 + R2

In a single-bus system, data is transferred sequentially. In a multi-bus system, R1 and R2 can be supplied to the ALU through separate buses.

OUTPUT:

```

=====
                        PERFORMANCE ANALYSIS
=====

Single-Bus Cycles      : 6
Multi-Bus Cycles       : 5

Single-Bus Transfers   : 5
Multi-Bus Transfers    : 5

Speedup                : 1.20x
Cycle Reduction        : 16.67%
Single-Bus Efficiency  : 0.83
Multi-Bus Efficiency   : 1.00

Interpretation:
The multi-bus organization reduces execution cycles because two operands can be transferred to the ALU simultaneously using separate buses.

=====
                        VALIDATION
=====

Single-Bus final memory value : 15
Multi-Bus final memory value  : 15

VALIDATION : PASS
Both architectures produced the same final result.
  
```

Comparison

Feature	Single Bus	Multi Bus
Number of buses	One	Two or more
Data transfer	Sequential	More parallel
Speed	Lower	Higher
Hardware	Simple	Complex
Bottleneck	More	Less

Analysis

The multi-bus system is faster because it allows more than one data transfer at a time. The single-bus system is simpler but can become a bottleneck.

Conclusion

Multi-bus organization provides better performance than single-bus organization because of parallel data transfer.

Q3. Five-Stage Instruction Pipeline

Aim

To simulate a five-stage instruction pipeline and compare it with a non-pipelined system.

Concept

The five pipeline stages are:

IF → ID → EX → MEM → WB

IF – Instruction Fetch

ID – Instruction Decode

EX – Execute

MEM – Memory Access

WB – Write Back

Example

Instructions

I1: ADD

I2: SUB

I3: AND

I4: OR

Pipeline Table

Cycle	I1	I2	I3	I4
1	IF			
2	ID	IF		
3	EX	ID	IF	
4	MEM	EX	ID	IF
5	WB	MEM	EX	ID
6		WB	MEM	EX
7			WB	MEM

OUTPUT:

```

=====
                        PERFORMANCE ANALYSIS
=====

Single-Bus Cycles      : 6
Multi-Bus Cycles       : 5

Single-Bus Transfers   : 5
Multi-Bus Transfers    : 5

Speedup                 : 1.20x
Cycle Reduction         : 16.67%
Single-Bus Efficiency   : 0.83
Multi-Bus Efficiency    : 1.00

Interpretation:
The multi-bus organization reduces execution cycles because two operands can be transferred to the ALU simultaneously using separate buses.

=====
                        VALIDATION
=====

Single-Bus final memory value : 15
Multi-Bus final memory value  : 15

VALIDATION : PASS
Both architectures produced the same final result.

```

Calculation

For 4 instructions and 5 stages:

Non-pipelined:

$$4 \times 5 = 20 \text{ cycles}$$

Pipelined:

$$5 + 4 - 1 = 8 \text{ cycles}$$

Speedup:

$$20 / 8 = 2.5$$

Analysis

Pipelining allows different instructions to use different stages at the same time. Therefore, more instructions can be completed in less time.

Conclusion

The pipelined processor provides better instruction throughput than the non-pipelined processor.

Q4. Pipeline Hazards

Aim

To study data, control, and structural hazards in a pipeline and understand methods to reduce their effects.

Concept

A pipeline hazard occurs when an instruction cannot execute in its normal clock cycle.

1. Data Hazard

I1: ADD R1,R2,R3

I2: SUB R4,R1,R5

I2 needs the result of I1.

Solution: Data forwarding or stalling.

2. Control Hazard

I1: BEQ R1,R2,Target

The processor does not immediately know which instruction should execute next.

Solution: Branch prediction.

3. Structural Hazard

Two instructions need the same hardware resource at the same time.

Solution: Add more resources or insert a stall.

Hazard Handling

Hazard Solution

Data Forwarding / Stalling

Control Branch Prediction

Structural Additional Hardware / Stalling

Analysis

OUTPUT:

```
--- Without Resource Scheduling ---
Resource conflict occurs.
Stalls introduced : 2
Total cycles      : 8

--- With Resource Scheduling ---
Instructions are scheduled so that the resource is not requested simultaneously.
Stalls introduced : 0
Total cycles      : 6

Speedup with scheduling : 1.33x

=====
                        PERFORMANCE SUMMARY
=====

Hazard                Baseline    Optimized    Speedup
-----
Data Hazard           7          5          1.40x
Control Hazard        8          6          1.33x
Structural Hazard     8          6          1.33x

=====
                        VALIDATION
=====

Data Hazard:
Forwarding removes unnecessary waiting.
VALIDATION : PASS

Control Hazard:
Branch prediction reduces branch penalty.
VALIDATION : PASS

Structural Hazard:
Resource scheduling prevents resource conflicts.
VALIDATION : PASS

Overall Validation : PASS
```

Hazards reduce pipeline performance because they can introduce extra clock cycles. Forwarding reduces data delays, while branch prediction reduces branch-related delays.

Conclusion

Proper hazard-handling techniques improve pipeline performance and reduce unnecessary stalls.

Q5. Superscalar, Out-of-Order and SMT

Aim

To study superscalar, out-of-order, speculative execution, and SMT techniques used in modern processors.

Concept

A superscalar processor can execute more than one instruction in a clock cycle.

Example:

Cycle 1 → I1, I2

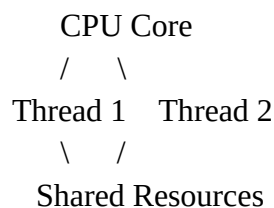
Cycle 2 → I3, I4

Cycle 3 → I5, I6

Out-of-order execution allows an independent instruction to execute before an earlier instruction if the earlier one is waiting.

Speculative execution executes instructions based on a predicted branch result.

SMT (Simultaneous Multithreading) allows multiple threads to use the processor at the same time.



Comparison

Technique	Main Benefit
Superscalar	Multiple instructions/cycle
Out-of-order	Better resource utilization
Speculation	Reduces branch delay
SMT	Executes multiple threads

Example

For 10 independent instructions:

Single issue = approximately 10 cycles

2-wide issue = approximately 5 cycles

The theoretical speedup is approximately:

$$10 / 5 = 2\times$$

OUTPUT:

```
=====
PS C:\Users\SANDY\Downloads\CA CO3 AT1> python experiment5.py
=====

      COMPUTER ARCHITECTURE - EXPERIMENT 5
    SUPERSCALAR, OUT-OF-ORDER AND SMT PROCESSING
=====

=====
                        SCALAR IN-ORDER
=====
Cycle 1: Execute I1: ADD R1,R2,R3
Cycle 2: Execute I2: SUB R4,R5,R6
Cycle 3: Execute I3: MUL R7,R1,R8
Cycle 4: Execute I4: ADD R9,R4,R10
Cycle 5: Execute I5: SUB R11,R7,R12
Cycle 6: Execute I6: ADD R13,R9,R14
Cycle 7: Execute I7: MUL R15,R11,R16
Cycle 8: Execute I8: ADD R17,R13,R18

Scalar cycles = 8

=====
                        2-WAY SUPERSCALAR IN-ORDER
=====
Cycle 1: Execute I1: ADD R1,R2,R3 | Execute I2: SUB R4,R5,R6
Cycle 2: Execute I3: MUL R7,R1,R8 | Execute I4: ADD R9,R4,R10
Cycle 3: Execute I5: SUB R11,R7,R12 | Execute I6: ADD R13,R9,R14
Cycle 4: Execute I7: MUL R15,R11,R16 | Execute I8: ADD R17,R13,R18

Superscalar cycles = 4
```

Analysis

These techniques increase parallelism and improve processor throughput. However, actual performance is affected by dependencies, branch misprediction, memory delays, and limited hardware resources.

Conclusion

Superscalar execution, out-of-order execution, speculation, and SMT improve processor performance by allowing more instructions or threads to execute in parallel.